

✓ Lab 7 – Chroniques Météo de Melbourne

Visualisation interactive avec Bokeh

Téléchargez le dataset depuis Kaggle ou utilisez le lien ci-dessous, puis uploadez-le dans Colab.

```
# — 0. Installation & imports —————
!pip install bokeh --quiet

import pandas as pd
import numpy as np
from bokeh.plotting import figure, show, output_notebook
from bokeh.models import (
    ColumnDataSource, HoverTool, DatetimeTickFormatter,
    DateRangeSlider, CustomJS, BoxAnnotation, Legend,
    Whisker, Band
)
from bokeh.layouts import column, gridplot
from bokeh.transform import factor_cmap
from bokeh.palettes import Category10, Viridis256, Spectral6
from bokeh.io import push_notebook

output_notebook()
print('Bokeh prêt ✓')
```

Bokeh prêt ✓

```
# — 1. Chargement et nettoyage du dataset —————
# Option A - depuis Google Drive (décommentez si besoin) :
# from google.colab import drive
# drive.mount('/content/drive')
# filepath = '/content/drive/MyDrive/datasets/daily-minimum-temperatures-in-melbourne.csv'

# Option B - upload manuel dans Colab :
# from google.colab import files
# uploaded = files.upload() # choisissez le CSV
# filepath = list(uploaded.keys())[0]

# Option C - téléchargement direct depuis GitHub (dataset public)
url = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/daily-min-temperatures.csv'
df = pd.read_csv(url, header=0, names=['Date', 'Temperature'])

# --- Nettoyage ---
# Supprimer les lignes où Temperature vaut '?'
df['Temperature'] = df['Temperature'].astype(str).str.replace('?', '', regex=False)
df['Temperature'] = pd.to_numeric(df['Temperature'], errors='coerce')
df.dropna(subset=['Temperature'], inplace=True)

# Conversion de la colonne Date en datetime
df['Date'] = pd.to_datetime(df['Date'])
df.sort_values('Date', inplace=True)
df.reset_index(drop=True, inplace=True)

print(f'Dataset chargé : {len(df)} lignes')
print(f'Période : {df["Date"].min().date()} → {df["Date"].max().date()}')
df.head()
```

Dataset chargé : 3650 lignes
Période : 1981-01-01 → 1990-12-31

	Date	Temperature
0	1981-01-01	20.7
1	1981-01-02	17.9
2	1981-01-03	18.8
3	1981-01-04	14.6
4	1981-01-05	15.8

Étapes suivantes : [Générer du code avec df](#) [New interactive sheet](#)

✓ Question 1 – Graphique linéaire de série temporelle de base

```

source_q1 = ColumnDataSource(df)

p1 = figure(
    title='Daily Minimum Temperatures',
    x_axis_label='Date',
    y_axis_label='Temperature (°C)',
    x_axis_type='datetime',
    width=900, height=400,
    tools='pan,wheel_zoom,reset,save'
)

p1.line('Date', 'Temperature', source=source_q1,
        line_color='steelblue', line_width=1.2, alpha=0.8)

p1.add_tools(HoverTool(
    tooltips=[
        ('Date', '@Date{%F}'),
        ('Température', '@Temperature{0.1f} °C')
    ],
    formatters={'@Date': 'datetime'},
    mode='vline'
))

p1.xaxis.formatter = DatetimeTickFormatter(months='%b %Y', years='%Y')
p1.title.text_font_size = '14pt'

show(p1)

```

Question 2 – Moyenne mobile sur 30 jours

```

df['Rolling_Avg'] = df['Temperature'].rolling(window=30, center=True).mean()

source_q2 = ColumnDataSource(df)

p2 = figure(
    title='Daily Minimum Temperatures + Rolling Average (30 days)',
    x_axis_label='Date',
    y_axis_label='Temperature (°C)',
    x_axis_type='datetime',
    width=900, height=400,
    tools='pan,wheel_zoom,reset,save'
)

r_orig = p2.line('Date', 'Temperature', source=source_q2,
                 line_color='lightblue', line_width=1, alpha=0.6,
                 legend_label='Température originale')

r_avg = p2.line('Date', 'Rolling_Avg', source=source_q2,
                line_color='firebrick', line_width=2.5,
                legend_label='Moyenne mobile 30 j')

p2.add_tools(HoverTool(
    renderers=[r_avg],
    tooltips=[
        ('Date', '@Date{%F}'),
        ('Température', '@Temperature{0.1f} °C'),
        ('Moy. mobile', '@Rolling_Avg{0.1f} °C')
    ],

```

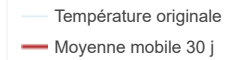
```

        formatters={'@Date': 'datetime'},
        mode='vline'
    ))

p2.xaxis.formatter = DatetimeTickFormatter(months='%b %Y', years='%Y')
p2.legend.location = 'top_right'
p2.legend.click_policy = 'hide'

show(p2)

```



Question 3 – Box plots mensuels

```

month_names = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
               'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']

df['Month'] = df['Date'].dt.month
df['MonthName'] = df['Date'].dt.strftime('%b')

# Stats par mois
stats_m = df.groupby('Month')['Temperature'].describe(percentiles=[.25, .5, .75])
stats_m.columns = ['count', 'mean', 'std', 'min', 'q1', 'median', 'q3', 'max']
stats_m['MonthName'] = month_names
stats_m['iqr'] = stats_m['q3'] - stats_m['q1']
stats_m['upper'] = (stats_m['q3'] + 1.5 * stats_m['iqr']).clip(upper=stats_m['max'])
stats_m['lower'] = (stats_m['q1'] - 1.5 * stats_m['iqr']).clip(lower=stats_m['min'])

source_m = ColumnDataSource(stats_m)

p3 = figure(
    title='Monthly Temperature Distribution (Box Plot)',
    x_range=month_names,
    x_axis_label='Month',
    y_axis_label='Temperature (°C)',
    width=900, height=450,
    tools='pan,wheel_zoom,reset,save'
)

# Moustaches
p3.segment('MonthName', 'upper', 'MonthName', 'q3', source=source_m, line_color='black')
p3.segment('MonthName', 'lower', 'MonthName', 'q1', source=source_m, line_color='black')

# Boîtes
p3.vbar('MonthName', 0.7, 'median', 'q3', source=source_m,
        fill_color='steelblue', line_color='black')
p3.vbar('MonthName', 0.7, 'q1', 'median', source=source_m,
        fill_color='lightblue', line_color='black')

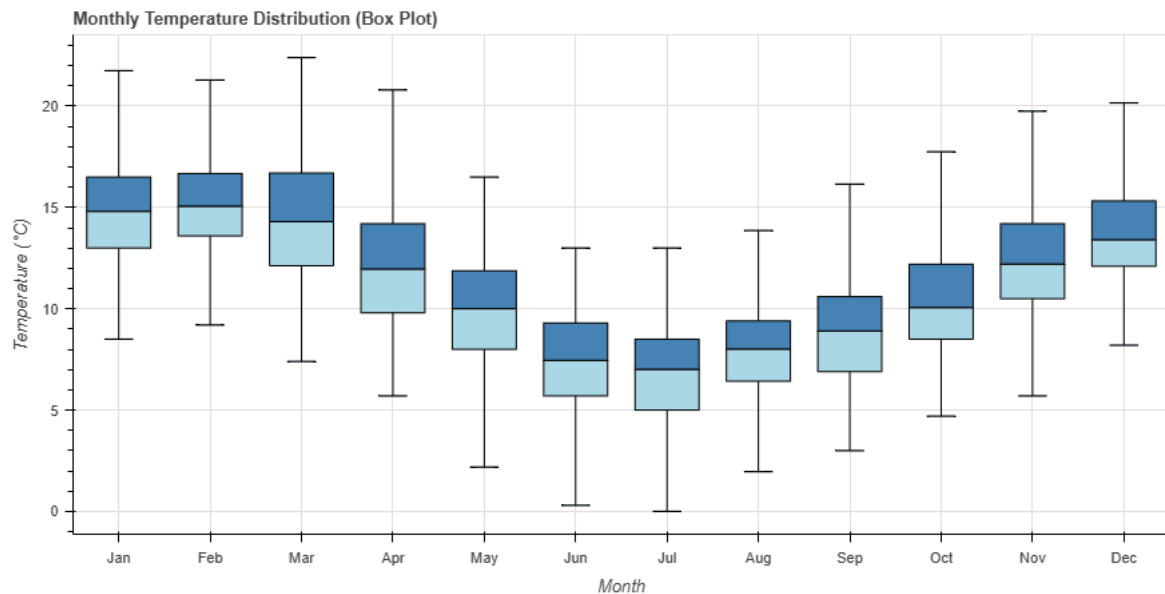
# Caps
p3.rect('MonthName', 'upper', 0.3, 0.01, source=source_m, line_color='black')
p3.rect('MonthName', 'lower', 0.3, 0.01, source=source_m, line_color='black')

p3.add_tools(HoverTool(tooltips=[
    ('Mois', '@MonthName'),
    ('Min', '@min{0.1f} °C'),
    ('Q1', '@q1{0.1f} °C'),
    ('Médiane', '@median{0.1f} °C'),
    ('Q3', '@q3{0.1f} °C'),
    ('Max', '@max{0.1f} °C')
]))

```

```
)))
```

```
show(p3)
```



Question 4 – Box plots annuels avec colormap

```
from bokeh.transform import linear_cmap
from bokeh.models import ColorBar, LinearColorMapper

df['Year'] = df['Date'].dt.year
years = sorted(df['Year'].unique())
year_strs = [str(y) for y in years]

stats_y = df.groupby('Year')['Temperature'].describe(percentiles=[.25, .5, .75])
stats_y.columns = ['count', 'mean', 'std', 'min', 'q1', 'median', 'q3', 'max']
stats_y['iqr'] = stats_y['q3'] - stats_y['q1']
stats_y['upper'] = (stats_y['q3'] + 1.5 * stats_y['iqr']).clip(upper=stats_y['max'])
stats_y['lower'] = (stats_y['q1'] - 1.5 * stats_y['iqr']).clip(lower=stats_y['min'])

stats_y = stats_y.reset_index()
stats_y['Year'] = stats_y['Year'].astype(str)

source_y = ColumnDataSource(stats_y)

mapper = LinearColorMapper(
    palette=Viridis256,
    low=stats_y['median'].min(),
    high=stats_y['median'].max()
)

p4 = figure(
    title='Annual Temperature Distribution (Box Plot + Color Map)',
    x_range=year_strs,
    x_axis_label='Year',
    y_axis_label='Temperature (°C)',
    width=900, height=450,
    tools='pan,wheel_zoom,reset,save'
)

p4.segment('Year', 'upper', 'Year', 'q3', source=source_y, line_color='black')
p4.segment('Year', 'lower', 'Year', 'q1', source=source_y, line_color='black')

p4.vbar('Year', 0.7, 'median', 'q3', source=source_y,
        fill_color={'field': 'median', 'transform': mapper}, line_color='black')
p4.vbar('Year', 0.7, 'q1', 'median', source=source_y,
        fill_color={'field': 'median', 'transform': mapper}, line_color='black', alpha=0.6)

p4.rect('Year', 'upper', 0.4, 0.01, source=source_y, line_color='black')
p4.rect('Year', 'lower', 0.4, 0.01, source=source_y, line_color='black')

color_bar = ColorBar(color_mapper=mapper, label_standoff=12, width=12, title='Médiane (°C)')
p4.add_layout(color_bar, 'right')

p4.add_tools(HoverTool(tooltips=[
    ('Année', '@Year'),
```

```

        ('Min', '@min{0.1f} °C'),
        ('Q1', '@q1{0.1f} °C'),
        ('Médiane', '@median{0.1f} °C'),
        ('Q3', '@q3{0.1f} °C'),
        ('Max', '@max{0.1f} °C'),
        ('Moyenne', '@mean{0.1f} °C')
    ]))

p4.xaxis.major_label_orientation = 0.5
show(p4)

```

Question 5 – Sélection interactive de la plage temporelle

```

from bokeh.models import DateRangeSlider, CustomJS

source_full = ColumnDataSource(df)
source_filtered = ColumnDataSource(df.copy())

p5 = figure(
    title='Daily Minimum Temperatures - Interactive Range Selection',
    x_axis_label='Date',
    y_axis_label='Temperature (°C)',
    x_axis_type='datetime',
    width=900, height=380,
    tools='pan,wheel_zoom,reset,save'
)

p5.line('Date', 'Temperature', source=source_filtered,
        line_color='steelblue', line_width=1.2, alpha=0.8)
p5.line('Date', 'Rolling_Avg', source=source_filtered,
        line_color='firebrick', line_width=2, alpha=0.9)

p5.add_tools(HoverTool(
    tooltips=[
        ('Date', '@Date{%F}'),
        ('Température', '@Temperature{0.1f} °C'),
        ('Moy. mobile', '@Rolling_Avg{0.2f} °C')
    ],
    formatters={'@Date': 'datetime'},
    mode='vline'
))
p5.xaxis.formatter = DatetimeTickFormatter(months='%b %Y', years='%Y')

# Slider de plage de dates
date_range_slider = DateRangeSlider(
    title='Plage de dates',
    start=df['Date'].min(),
    end=df['Date'].max(),
    value=(df['Date'].min(), df['Date'].max()),
    step=1,
    width=880
)

# Callback JS pour filtrer les données
callback = CustomJS(args=dict(source_full=source_full,

```

```

        source_filtered=source_filtered,
        slider=date_range_slider), code="""
const start = slider.value[0];
const end   = slider.value[1];
const full  = source_full.data;
const filt  = {Date: [], Temperature: [], Rolling_Avg: []};

for (let i = 0; i < full['Date'].length; i++) {
  const d = full['Date'][i];
  if (d >= start && d <= end) {
    filt['Date'].push(d);
    filt['Temperature'].push(full['Temperature'][i]);
    filt['Rolling_Avg'].push(full['Rolling_Avg'][i]);
  }
}
source_filtered.data = filt;
source_filtered.change.emit();
""")

date_range_slider.js_on_change('value', callback)

layout_q5 = column(p5, date_range_slider)
show(layout_q5)

```

Plage de dates: 01 Jan 1981 .. 31 Dec 1990



Question 6 – Décomposition de série temporelle

```

# Rééchantillonnage mensuel
df_monthly = df.set_index('Date').resample('ME')['Temperature'].mean().reset_index()
df_monthly.columns = ['Date', 'Temp_Monthly']

# Tendence : moyenne mobile 12 mois
df_monthly['Trend'] = df_monthly['Temp_Monthly'].rolling(window=12, center=True).mean()

# Saisonnalité = données mensuelles - tendance
df_monthly['Seasonality'] = df_monthly['Temp_Monthly'] - df_monthly['Trend']

src = ColumnDataSource(df_monthly)

hover_monthly = HoverTool(
  tooltips=[('Date', '@Date{%b %Y}'), ('Valeur', '@Temp_Monthly{0.2f} °C')],
  formatters={'@Date': 'datetime'}, mode='vline'
)
hover_trend = HoverTool(
  tooltips=[('Date', '@Date{%b %Y}'), ('Tendance', '@Trend{0.2f} °C')],
  formatters={'@Date': 'datetime'}, mode='vline'
)
hover_season = HoverTool(
  tooltips=[('Date', '@Date{%b %Y}'), ('Saisonnalité', '@Seasonality{0.2f} °C')],
  formatters={'@Date': 'datetime'}, mode='vline'
)

kw = dict(x_axis_type='datetime', width=900, height=250,
  tools='pan,wheel_zoom,reset,save')

pa = figure(title='Données mensuelles', y_axis_label='Temp. (°C)', **kw)
pa.line('Date', 'Temp_Monthly', source=src, line_color='steelblue', line_width=1.5)

```

```
pa.add_tools(hover_monthly)
pa.xaxis.formatter = DatetimeTickFormatter(months='%b %Y', years='%Y')

pb = figure(title='Tendance (Moy. mobile 12 mois)', y_axis_label='Temp. (°C)',
            x_range=pa.x_range, **kw)
pb.line('Date', 'Trend', source=src, line_color='firebrick', line_width=2)
pb.add_tools(hover_trend)
pb.xaxis.formatter = DatetimeTickFormatter(months='%b %Y', years='%Y')

pc = figure(title='Saisonnalité', y_axis_label='Écart (°C)',
            x_range=pa.x_range, x_axis_label='Date', **kw)
pc.line('Date', 'Seasonality', source=src, line_color='green', line_width=1.5)
pc.add_tools(hover_season)
pc.xaxis.formatter = DatetimeTickFormatter(months='%b %Y', years='%Y')

layout_q6 = gridplot([[pa], [pb], [pc]])
show(layout_q6)
```

